

Chapter 18

QR Factorization

*Any orthogonal matrix can be written as the product of reflector matrices.
Thus the class of reflections is rich enough for all occasions
and yet each member is characterized by a single vector
which serves to describe its mirror.*

- BERESFORD N. PARLETT, *The Symmetric Eigenvalue Problem* (1980)

*A key observation for understanding the numerical properties of the
modified Gram–Schmidt algorithm is that it can be interpreted as
Householder QR factorization applied to the matrix A
augmented with a square matrix of zero elements on top.
These two algorithms are not only mathematically . . .
but also numerically equivalent.
This key observation, apparently by Charles Sheffield,
was relayed to the author in 1968 by Gene Golub.*

- AKE BJÖRCK, *Numerics of Gram-Schmidt Orthogonalization* (1994)

*The great stability of unitary transformations in numerical analysis
springs from the fact that both the ℓ_2 -norm
and the Frobenius norm are unitarily invariant.
This means in practice that even when rounding errors are made,
no substantial growth takes place in the
norms of the successive transformed matrices.*

- J. H. WILKINSON,
*Error Analysis of Transformations Based on the
Use of Matrices of the Form $I - 2ww^T$* (1965)

The QR factorization is a versatile computational tool that finds use in linear equation, least squares and eigenvalue problems. It can be computed in several ways, including by the use of Householder transformations and Givens rotations, and by the Gram–Schmidt method. We explore the numerical properties of all three methods in this chapter. We also examine the use of iterative refinement on a linear system solved with a QR factorization and consider the inherent sensitivity of the QR factorization.

18.1. Householder Transformations

A Householder matrix (also known as a Householder transformation, or Householder reflector) is a matrix of the form

$$P = I - \frac{2}{v^T v} v v^T, \quad 0 \neq v \in \mathbf{R}^n.$$

It enjoys the properties of symmetry and orthogonality, and, consequently, is involutory ($P^2 = I$). The application of P to a vector yields

$$Px = x - \left(\frac{2v^T x}{v^T v} \right) v.$$

Figure 18.1 illustrates this formula and makes it clear why P is sometimes called a Householder reflector: it reflects x about the hyperplane $\text{span}(v)^\perp$.

Householder matrices are powerful tools for introducing zeros into vectors. Consider the question “given x and y can we find a Householder matrix P such that $Px = y$?” Since P is orthogonal we clearly require that $\|x\|_2 = \|y\|_2$. Now

$$Px = y \iff x - 2 \left(\frac{v^T x}{v^T v} \right) v = y,$$

and this last equation has the form $av = x - y$ for some a . But P is independent of the scaling of v , so we can set $a = 1$.

With $v = x - y$ we have

$$v^T v = x^T x + y^T y - 2x^T y,$$

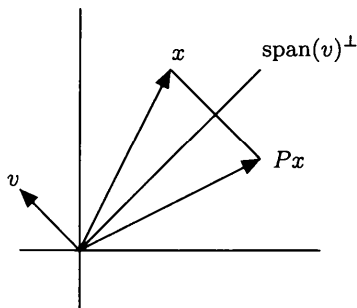
and, since $x^T x = y^T y$,

$$v^T x = x^T x - y^T x = \frac{1}{2} v^T v.$$

Therefore

$$Px = x - v = y,$$

as required. We conclude that, provided $\|x\|_2 = \|y\|_2$, we can find a Householder matrix P such that $Px = y$. (Strictly speaking, we have to exclude the case $x = y$, which would require $v = 0$, making P undefined).

Figure 18.1. Householder matrix P times vector x .

Normally we choose y to have a special pattern of zeros. The usual choice is $y = \sigma e_1$ where $\sigma = \pm \|x\|_2$, which yields the maximum number of zeros in y . Then

$$v = x - y = x - \sigma e_1.$$

We choose $\text{sign}(\sigma) = -\text{sign}(x_1)$ to avoid cancellation in the expression for v_1 .

18.2. QR Factorization

A QR factorization of $A \in \mathbb{R}^{m \times n}$ with $m \geq n$ is a factorization

$$A = QR = [Q_1 \quad Q_2] \begin{bmatrix} R_1 \\ 0 \end{bmatrix} = Q_1 R_1,$$

where $Q \in \mathbb{R}^{m \times m}$ is orthogonal and $R_1 \in \mathbb{R}^{n \times n}$ is upper triangular. The matrix R is called upper trapezoidal, since the term triangular applies only to square matrices. Depending on the context, either the full factorization $A = QR$ or the “economy size” version $A = Q_1 R_1$ can be called a QR factorization. A quick existence proof of the QR factorization is provided by the Cholesky factorization: if A has full rank and $A^T A = R^T R$ is a Cholesky factorization, then $A = AR^{-1} \cdot R$ is a QR factorization. The QR factorization is unique if A has full rank and we require R to have positive diagonal elements ($A = QD \cdot DR$ is a QR factorization for any $D = \text{diag}(\pm 1)$).

The QR factorization can be computed by premultiplying the given matrix by a suitably chosen sequence of Householder matrices. The process is

illustrated for a generic 4×3 matrix as follows:

$$A = \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \xrightarrow{P_1} \left[\begin{array}{c|cc} \times & \times & \times \\ \hline 0 & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \end{array} \right] \xrightarrow{P_2} \left[\begin{array}{cc|c} \times & \times & \times \\ \hline 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & \times \end{array} \right] \xrightarrow{P_3} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & 0 \end{bmatrix} = R.$$

The general process is adequately described by the k th stage of the reduction to triangular form. With $A_1 = A$ we have, at the start of the k th stage,

$$A_k = \begin{bmatrix} R_{k-1} & z_k & B_k \\ 0 & x_k & C_k \end{bmatrix}, \quad R_{k-1} \in \mathbb{R}^{(k-1) \times (k-1)}, \quad x_k \in \mathbb{R}^{m-k+1}, \quad (18.1)$$

where R_{k-1} is upper triangular. Choose a Householder matrix $\tilde{P}_k$ such that $\tilde{P}_k x_k = \sigma e_1$ and embed $\tilde{P}_k$ into an $m \times m$ matrix

$$P_k = \begin{bmatrix} I_{k-1} & 0 \\ 0 & \tilde{P}_k \end{bmatrix}. \quad (18.2)$$

Then let $A_{k+1} = P_k A_k$. Overall, we obtain $R = P_n P_{n-1} \dots P_1 A =: Q^T A$ ($P_n = I$ if $m = n$).

To compute A_{k+1} we need to form $\tilde{P}_k C_k$. We can write

$$\tilde{P}_k C_k = (I - \beta v v^T) C_k = C_k - \beta v (v^T C_k), \quad \beta = 2/(v^T v),$$

which shows that the matrix product can be formed as a matrix–vector product followed by an outer product. This approach is much more efficient than forming $\tilde{P}_k$ explicitly and doing a matrix multiplication.

The overall cost of the Householder reduction to triangular form is $2n^2(m - n/3)$ flops. The explicit formation of Q requires a further $4(m^2 n - mn^2 + n^3/3)$ flops, but for many applications it suffices to leave Q in factored form.

18.3. Error Analysis of Householder Computations

It is well known that computations with Householder matrices are very stable. Wilkinson showed that the computation of a Householder vector, and the application of a Householder matrix to a given matrix, are both normwise stable, in the sense that the computed Householder vector is very close to

the exact one and the computed update is the exact update of a tiny normwise perturbation of the original matrix [1089, 1965, pp. 153–162, 236], [1090, 1965]. Wilkinson also showed that the Householder QR factorization algorithm is normwise backward stable [1089, p. 236]. In this section we give a combined componentwise and normwise error analysis of Householder matrix computations. The componentwise bounds provide extra information over the normwise ones that is essential in certain applications (for example, the analysis of iterative refinement).

Lemma 18.1. *Let $x \in \mathbb{R}^n$. Consider the following construction of $\beta \in \mathbb{R}$ and $v \in \mathbb{R}^n$ such that $Px = \sigma e_1$, where $P = I - \beta vv^T$ is a Householder matrix with $\beta = 2/(v^T v)$:*

$$\begin{aligned} v &= x \\ s &= \text{sign}(x_1)\|x\|_2 \quad \% \sigma = -s \\ v_1 &= v_1 + s \\ \beta &= 1/(sv_1) \end{aligned}$$

In floating point arithmetic the computed $\hat{\beta}$ and $\hat{v}$ satisfy $\hat{v}(2:n) = v(2:n)$ and

$$\begin{aligned} \hat{\beta} &= \beta(1 + \theta_{4n+8}), \\ \hat{v}_1 &= v_1(1 + \theta_{n+2}), \end{aligned}$$

where $|\theta_k| \leq \gamma_k$.

Proof. We sketch the proof. Each occurrence of δ denotes a different number bounded by $|\delta| \leq u$. We compute $fl(x^T x) = (1 + \theta_n)x^T x$, and then $fl(\|x\|_2) = (1 + \delta)(1 + \theta_n)^{1/2}(x^T x)^{1/2} = (1 + \theta_{n+1})\|x\|_2$ (the latter term $1 + \theta_{n+1}$ is suboptimal, but our main aim is to keep the analysis simple). Hence $\hat{s} = (1 + \theta_{n+1})s$.

For notational convenience, define $w = v_1 + s$. We have $\hat{w} = (v_1 + \hat{s})(1 + \delta) = w(1 + \theta_{n+2})$ (essentially because there is no cancellation in the sum). Hence

$$\begin{aligned} \hat{\beta} &= fl(1/(\hat{s}\hat{w})) = \frac{(1 + \delta)^2}{(1 + \theta_{n+1})s(1 + \theta_{n+2})w} \\ &= \frac{(1 + \delta)^2}{(1 + \theta_{2n+3})sw} = (1 + \theta_{4n+8})\beta. \quad \square \end{aligned}$$

For convenience we will henceforth write Householder matrices in the form $I - vv^T$, which requires $\|v\|_2 = \sqrt{2}$ and amounts to redefining $v := \sqrt{\beta}v$

and $\beta := 1$ in the representation of Lemma 18.1. We can then write, using Lemma 18.1,

$$\hat{v} = v + \Delta v, \quad |\Delta v| \leq \gamma_{cm}|v| \quad (v \in \mathbf{R}^m, \quad \|v\|_2 = \sqrt{2}), \quad (18.3)$$

where, as required for the next two results, the dimension is now m . Here, we have introduced the generic constant

$$\gamma_{cm} = \frac{cmu}{1 - cmu},$$

in which c denotes a small integer constant whose exact value is unimportant. We will make frequent use of γ_{cm} in the rest of this chapter, because it is not worthwhile to evaluate the integer constants in our bounds explicitly. Because we are not “chasing constants” we can afford to be somewhat cavalier and freely use inequalities such as

$$\gamma_{cm} + \gamma_m + \gamma_{cm}\gamma_m + \gamma_{cm}^2 \leq \gamma'_{cm}$$

(recall Lemma 3.3), where we signify by the prime that the constant c on the right-hand side is different from that on the left.

The next result describes the application of a Householder matrix to a vector, and is the basis of all the subsequent analysis. In the applications of interest P is defined as in Lemma 18.1, but we will allow P to be an arbitrary Householder matrix. Thus v is an arbitrary, normalized vector, and the only assumption we make is that the computed $\hat{v}$ satisfies (18.3).

Lemma 18.2. *Let $b \in \mathbf{R}^m$ and consider the computation of $y = \hat{P}b = (I - \hat{v}\hat{v}^T)b = b - \hat{v}(\hat{v}^T b)$, when $\hat{v} \in \mathbf{R}^m$ satisfies (18.3): The computed $\hat{y}$ satisfies*

$$\hat{y} = (P + \Delta P)b, \quad \|\Delta P\|_F \leq \gamma_{cm},$$

where $P = I - vv^T$.

Proof. (Cf. the proof of Lemma 3.8.) We have

$$\hat{w} := fl(\hat{v}(\hat{v}^T b)) = (\hat{v} + \Delta\hat{v})(\hat{v}^T(b + \Delta b)),$$

where $|\Delta\hat{v}| \leq u|\hat{v}|$ and $|\Delta b| \leq \gamma_m|b|$. Hence

$$\hat{w} = (v + \Delta v + \Delta\hat{v})(v + \Delta v)^T(b + \Delta b) =: v(v^T b) + \Delta w,$$

where $|\Delta w| \leq \gamma_{cm}|v||v^T||b|$. Then

$$\hat{y} = fl(b - \hat{w}) = b - v(v^T b) - \Delta w + \Delta y_1, \quad |\Delta y_1| \leq u|b - \hat{w}|.$$

We have

$$|-\Delta w + \Delta y_1| \leq u|b| + \gamma'_{cm}|v||v^T||b|.$$

Hence $\hat{y} = Pb + \Delta y$, where $\|\Delta y\|_2 \leq \gamma''_{cm} \|b\|_2$. But then $\hat{y} = (P + \Delta P)b$, where $\Delta P = \Delta y b^T / b^T b$ satisfies $\|\Delta P\|_F = \|\Delta y\|_2 / \|b\|_2 \leq \gamma''_{cm}$. $\square$

Next, we consider a sequence of Householder transformations applied to a matrix. Again, each Householder matrix is arbitrary and need have no connection to the matrix to which it is being applied. In the cases of interest, the Householder matrices P_k have the form (18.2), and so are of ever-decreasing effective dimension, but to exploit this property would not lead to any significant improvement in the bounds.

For the remaining results in this section, we make the (reasonable) assumption that an inequality of the form

$$mr\gamma_{cm} < \frac{1}{2} \quad (18.4)$$

holds, where r is the number of Householder transformations and it is implicit that G and H denote nonnegative matrices.

Lemma 18.3. *Consider the sequence of transformations*

$$A_{k+1} = P_k A_k, \quad k = 1:r,$$

where $A_1 = A \in \mathbf{R}^{m \times n}$ and $P_k = I - v_k v_k^T \in \mathbf{R}^{m \times m}$ is a Householder matrix. Assume that the transformations are performed using computed Householder vectors $\hat{v}_k \approx v_k$ that satisfy (18.3). The computed matrix A_{r+1} satisfies

$$\hat{A}_{r+1} = Q^T (A + \Delta A), \quad (18.5)$$

where $Q^T = P_r P_{r-1} \dots P_1$ and ΔA satisfies the normwise and componentwise bounds

$$\begin{aligned} \|\Delta A\|_F &\leq r\gamma_{cm} \|A\|_F, \\ |\Delta A| &\leq mr\gamma_{cm} G|A|, \quad \|G\|_F = 1. \end{aligned}$$

(In fact, we can take $G = m^{-1} e e^T$, where $e = [1, 1, \dots, 1]^T$.) In the special case $n = 1$, so that $A \equiv a$, we have $\hat{a}^{(r+1)} = (Q + \Delta Q)^T a$ with $\|\Delta Q\|_F \leq r\gamma_{cm}$.

Proof. First, we consider the j th column of A , a_j , which undergoes the transformations $a_j^{(r+1)} = P_r \dots P_1 a_j$. By Lemma 18.2 we have

$$\hat{a}_j^{(r+1)} = (P_r + \Delta P_r) \dots (P_1 + \Delta P_1) a_j,$$

where each ΔP_k depends on j and satisfies $\|\Delta P_k\|_F \leq \gamma_{cm}$. Using Lemma 3.6 we obtain

$$\begin{aligned} \hat{a}_j^{(r+1)} &= Q^T (a_j + d_j), \\ \|d_j\|_2 &\leq ((1 + \gamma_{cm})^r - 1) \|a_j\|_2 \leq \frac{r\gamma_{cm}}{1 - r\gamma_{cm}} \|a_j\|_2 = r\gamma'_{cm} \|a_j\|_2, \end{aligned} \quad (18.6)$$

using Lemma 3.1 and assumption (18.4). Hence ΔA in (18.5) satisfies

$$\|\Delta A\|_F^2 = \sum_{j=1}^n \|d_j\|_2^2 \leq r^2 \gamma_{cm}'^2 \sum_{j=1}^n \|a_j\|_2^2 = r^2 \gamma_{cm}'^2 \|A\|_F^2.$$

NOW, since $\|a_j\|_2 \leq \|a_j\|_1 = e^T |a_j|$, from (18.6) we have

$$|\Delta A| \leq r \gamma_{cm}' e(e^T |A|) =: m r \gamma_{cm}' G |A|,$$

where $\|G\|_F = 1$ (since $\|ee^T\|_F = m$). Finally, if $n = 1$, so that A is a column vector, then (as in the proof of Lemma 18.2) we can rewrite (18.5) as $\hat{a}^{(r+1)} = (Q + \Delta Q)^T a$, where $\Delta Q^T = (Q^T \Delta a) a^T / a^T a$ and $\|\Delta Q\|_F = \|\Delta a\|_2 / \|a\|_2 \leq r \gamma_{cm}$. $\square$

Note that the componentwise bound for ΔA in Lemma 18.3 does not imply the normwise one, because of the extra factor m in the componentwise bound. This is a nuisance, because it means we have to state both bounds in this and other analyses.

We now apply Lemma 18.3 to the computation of the QR factorization of a matrix.

Theorem 18.4. *Let $\hat{R} \in \mathbb{R}^{m \times n}$ be the computed upper trapezoidal QR factor of $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) obtained via the Householder QR algorithm. Then there exists an orthogonal $Q \in \mathbb{R}^{m \times m}$ such that*

$$A + \Delta A = Q \hat{R},$$

where $\|\Delta A\|_F \leq n \gamma_{cm} \|A\|_F$ and $|\Delta A| \leq m n \gamma_{cm} G |A|$, with $\|G\|_F = 1$. The matrix Q is given explicitly as $Q = (P_n P_{n-1} \dots P_1)^T$, where P_k is the Householder matrix that corresponds to the exact application of the k th step of the algorithm to A_k .

Proof. This is virtually a direct application of Lemma 18.3, with P_k defined as the Householder matrix that produces zeros below the diagonal in the k th column of the computed matrix A_k . one subtlety is that we do not explicitly compute the lower triangular elements of R , but rather set them to zero explicitly. However, it is easy to see that the conclusions of Lemmas 18.2 and 18.3 are still valid in these circumstances; the essential reason is that the elements of $\Delta P b$ in Lemma 18.2 that correspond to elements that are zeroed by the Householder matrix P are forced to be zero, and hence we can set the corresponding rows of ΔP to zero too, without compromising the bound on $\|\Delta P\|_F$. $\square$

Finally, we consider use of the QR factorization to solve a linear system. Given a QR factorization of a nonsingular matrix $A \in \mathbb{R}^{n \times n}$, a linear system $Ax = b$ can be solved by forming $Q^T b$ and then solving $Rx = Q^T b$.

From Theorem 18.4, the computed $\hat{R}$ is guaranteed to be nonsingular if $\kappa_2(A)n^{3/2}\gamma_{cn} < 1$. We give only componentwise bounds.

Theorem 18.5. *Let $A \in \mathbb{R}^{n \times n}$ be nonsingular. Suppose we solve the system $Ax = b$ with the aid of a QR factorization computed by the Householder algorithm. The computed $\hat{x}$ satisfies*

$$(A + \Delta A)\hat{x} = b + \Delta b,$$

where

$$|\Delta A| \leq n^2\gamma_{cn}G|A|, \quad |\Delta b| \leq n^2\gamma_{cn}G|b|, \quad \|G\|_F = 1.$$

Proof. By Theorem 18.4, the computed upper triangular factor $\hat{R}$ satisfies $A + \Delta A = QR$ with $|\Delta A| \leq n^2\gamma_{cn}G_1|A|$ and $\|G_1\|_F = 1$. By Lemma 18.3, the computed transformed right-hand side satisfies $\hat{c} = Q^T(b + \Delta b)$ with $|\Delta b| \leq n^2\gamma_{cn}G_1|b|$. Importantly, the same orthogonal matrix Q appears in the equations involving $\hat{R}$ and $\hat{c}$.

By Theorem 8.5, the computed solution $\hat{x}$ to the triangular system $\hat{R}\hat{x} = \hat{c}$ satisfies

$$(\hat{R} + \Delta R)\hat{x} = \hat{c}, \quad |\Delta R| \leq \gamma_n|\hat{R}|.$$

Premultiplying by Q yields

$$(A + \Delta A + Q\Delta R)\hat{x} = b + \Delta b,$$

that is, $(A + \overline{\Delta A})\hat{x} = b + \Delta b$, where $\overline{\Delta A} = \Delta A + Q\Delta R$. Using $\hat{R} = Q^T(A + \Delta A)$ we have

$$\begin{aligned} |\overline{\Delta A}| &\leq n^2\gamma_{cn}G_1|A| + \gamma_n|Q||\hat{R}| \\ &\leq n^2\gamma_{cn}G_1|A| + \gamma_n|Q||Q^T|(I + n^2\gamma_{cn}G_1)|A| \\ &=: n^2\gamma'_{cn}G|A|, \end{aligned}$$

where $G \geq G_1$ and $\|G\|_F = 1$. $\square$

The proof of Theorem 18.5 naturally leads to a result in which b is perturbed. However, we can easily modify the proof so that only A is perturbed: the trick is to use Lemma 18.3 to write $\hat{c} = (Q + \Delta Q)^T b$, where $\|\Delta Q\|_F \leq n\gamma_{cn}$, and to premultiply by $(Q + \Delta Q)^{-T}$ instead of Q in the middle of the proof. This leads to the result

$$(A + \Delta A)\hat{x} = b, \quad |\Delta A| \leq n^2\gamma_{cn}G|A|, \quad \|G\|_F = 1. \quad (18.7)$$

An interesting application of Theorem 18.5 is to iterative refinement, as explained in §18.6.

18.4. Aggregated Householder Transformations

In Chapter 12 we noted that LU factorization algorithms can be partitioned so as to express the bulk of the computation as matrix-matrix operations (level-3 BLAS). For computations with Householder transformations the same goal can be achieved by aggregating the transformations. This technique is widely used in LAPACK.

One form of aggregation is the “WY” representation of Bischof and Van Loan [105, 1987]. This involves representing the product $Q_r = P_r P_{r-1} \dots P_1$ of r Householder transformations $P_i = I - v_i v_i^T \in \mathbb{R}^{m \times m}$ (where $v_i^T v_i = 2$), in the form

$$Q_r = I + W_r Y_r^T, \quad W_r, Y_r \in \mathbb{R}^{m \times r}.$$

This can be done using the recurrence

$$W_1 = -v_1, \quad Y_1 = v_1, \quad W_i = [W_{i-1} \quad -v_i], \quad Y_i = [Y_{i-1} \quad Q_{i-1}^T v_i]. \quad (18.8)$$

Using the WY representation, a partitioned QR factorization can be developed as follows. Partition $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) as

$$A = [A_1 \quad B], \quad A_1 \in \mathbb{R}^{m \times r}, \quad (18.9)$$

and compute the Householder QR factorization of A_1 ,

$$P_r P_{r-1} \dots P_1 A_1 = \begin{bmatrix} R_1 \\ 0 \end{bmatrix}.$$

The product $P_r P_{r-1} \dots P_1 = I + W_r Y_r^T$ is accumulated using (18.8), as the P_i are generated, and then B is updated according to

$$B \leftarrow (I + W_r Y_r^T) B = B + W_r (Y_r^T B),$$

which involves only level-3 BLAS operations. The process is now repeated on the last $m - r$ rows of B .

When considering numerical stability, two aspects of the WY representation need investigating: its construction and its application. For the construction, we need to show that $Q_r := I + W_r Y_r^T$ satisfies

$$\|\widehat{Q}_r \widehat{Q}_r^T - I\|_2 \leq d_1 u, \quad (18.10)$$

$$\|\widehat{W}_r\|_2 \leq d_2, \quad \|\widehat{Y}_r\|_2 \leq d_3, \quad (18.11)$$

for modest constants d_1 , d_2 , and d_3 . Now

$$\widehat{Q}_i := I + [\widehat{W}_{i-1} \quad -\widehat{v}_i] \begin{bmatrix} \widehat{Y}_{i-1}^T \\ fl(\widehat{v}_i^T \widehat{Q}_{i-1}) \end{bmatrix} = \widehat{Q}_{i-1} - \widehat{v}_i fl(\widehat{v}_i^T \widehat{Q}_{i-1}).$$

But this last equation is essentially a standard multiplication by a Householder matrix, $\widehat{Q}_i = (I - \widehat{v}_i \widehat{v}_i^T) \widehat{Q}_{i-1}$, albeit with less opportunity for rounding errors. It follows from Lemma 18.3 that the near orthogonality of $\widehat{Q}_{i-1}$ is inherited by $\widehat{Q}_i$; the condition on $\widehat{Y}_r$ in (18.11) follows similarly and that on $\widehat{W}_r$ is trivial. Note that the condition (18.10) implies that

$$\widehat{Q}_r = U_r + \Delta U_r, \quad U_r^T U_r = I, \quad \|\Delta U_r\|_2 \leq d_1 u, \quad (18.12)$$

that is, $\widehat{Q}_r$ is close to an exactly orthogonal matrix (see Problem 18.13).

Next we consider the application of $\widehat{Q}_r$. Suppose we form $C = \widehat{Q}_r B = (I + \widehat{W}_r \widehat{Y}_r^T) B$ for the B in (18.9), so that

$$\widehat{C} = fl(B + fl(\widehat{W}_r (\widehat{Y}_r^T B))).$$

Analysing this level-3 BLAS-based computation using (18.12) and the very general assumption (12.3) on matrix multiplication (for the 2-norm), it is straightforward to show that

$$\begin{aligned} \widehat{C} &= U_r B + \Delta C = U_r (B + U_r^T \Delta C), \\ \|\Delta C\|_2 &\leq [1 + d_1 + d_2 d_3 (1 + c_1(r, m, n - r) \\ &\quad + c_1(m, r, n - r))] u \|B\|_2 + O(u^2). \end{aligned} \quad (18.13)$$

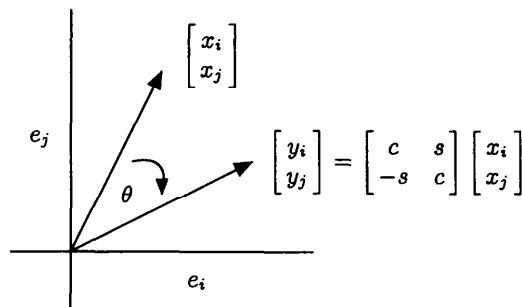
This result shows that the computed update is an exact orthogonal update of a perturbation of B , where the norm of the perturbation is bounded in terms of the error constants for the level-3 BLAS.

Two conclusions can be drawn. First, algorithms that employ the WY representation with conventional level-3 BLAS are as stable as the corresponding point algorithms. Second, the use of fast BLAS3 for applying the updates affects stability only through the constants in the backward error bounds. The same conclusions apply to the more storage-efficient compact WY representation of Schreiber and Van Loan [905, 1989], and the variation of Puglisi [848, 1992].

18.5. Givens Rotations

Another way to compute the QR factorization is with Givens rotations. A Givens rotation (or plane rotation) $G(i, j, \theta) \in \mathbf{R}^{n \times n}$ is equal to the identity matrix except that

$$G([i, j], [i, j]) = \begin{bmatrix} c & s \\ -s & c \end{bmatrix},$$

Figure 18.2. Givens rotation, $y = G(i, j, \theta)x$.

where $c = \cos\theta$ and $s = \sin\theta$. The multiplication $y = G(i, j, \theta)x$ rotates x through θ radians clockwise in the (i, j) plane; see Figure 18.2. Algebraically,

$$y_k = \begin{cases} x_k, & k \neq i, j, \\ cx_i + sx_j, & k = i, \\ -sx_i + cx_j, & k = j, \end{cases}$$

and so $y_j = 0$ if

$$s = \frac{x_j}{\sqrt{x_i^2 + x_j^2}}, \quad c = \frac{x_i}{\sqrt{x_i^2 + x_j^2}}. \quad (18.14)$$

Givens rotations are therefore useful for introducing zeros into a vector one at a time. Note that there is no need to work out the angle θ , since c and s in (18.14) are all that are needed to apply the rotation. In practice, we would scale the computation to avoid overflow (cf. §25.8).

To compute the QR factorization, Givens rotations are used to eliminate the elements below the diagonal in a systematic fashion. Various choices and orderings of rotations can be used; a natural one is illustrated as follows for a generic 4×3 matrix:

$$A = \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \xrightarrow{G_{34}} \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ 0 & \times & \times \end{bmatrix} \xrightarrow{G_{23}} \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \end{bmatrix} \xrightarrow{G_{12}} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \end{bmatrix} \xrightarrow{G_{34}} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \end{bmatrix} \xrightarrow{G_{23}} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & \times \end{bmatrix} \xrightarrow{G_{34}} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & \times \end{bmatrix}$$

$$\begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & 0 \end{bmatrix} = R.$$

The operation count for Givens QR factorization of a general $m \times n$ matrix ($m \geq n$) is $3n^2(m - n/3)$ flops, which is 50% more than that for Householder QR factorization. The main use of Givens rotations is to operate unstructured matrices—for example, to compute the QR factorization of a tridiagonal or Hessenberg matrix, or to carry out delicate zeroing in updating or downdating problems [470, 1989, §12.6].

Error analysis for Givens rotations is similar to that for Householder matrices—but a little easier. We omit the (straightforward) proof of the first result.

Lemma 18.6. *Let a Givens rotation $G(i, j, \theta)$ be instructed according to (18.14). The computed $\hat{c}$ and $\hat{s}$ satisfy*

$$\hat{c} = c(1 + \theta_4), \quad \hat{s} = s(1 + \theta'_4), \quad (18.15)$$

where $|\theta_4|, |\theta'_4| \leq \gamma_4$. $\square$

Lemma 18.7. *Let $x \in \mathbb{R}^m$ and consider the computation of $y = \hat{G}_{ij}x$, where $\hat{G}_{ij}$ is a computed Givens rotation in the (i, j) plane for which $\hat{c}$ and $\hat{s}$ satisfy (18.15). The computed $\hat{y}$ satisfies*

$$\hat{y} = (G_{ij} + \Delta G_{ij})x, \quad \|\Delta G_{ij}\|_F \leq \sqrt{2}\gamma_6,$$

where G_{ij} is an exact Givens rotation based on c and s in (18.15). All the rows of ΔG_{ij} except the i th and j th are zero.

Proof. The vector $\hat{y}$ differs from x only in elements i and j . We have

$$\hat{y}_i = fl(\hat{c}x_i + \hat{s}x_j) = cx_i(1 + \theta_6) + sx_j(1 + \theta'_6),$$

where $|\theta_6|, |\theta'_6| \leq \gamma_6$, and similarly for $\hat{y}_j$. Hence

$$|\hat{y} - G_{ij}x| \leq \gamma_6 |G_{ij}| |x|,$$

so that $\|\hat{y} - G_{ij}x\|_2 \leq \sqrt{2}\gamma_6 \|x\|_2$. We take $\Delta G_{ij} = (\hat{y} - G_{ij}x)x^T/x^T x$. $\square$

For the next result we need the notion of disjoint Givens rotations. Rotations $G_{i_1 j_1}, \dots, G_{i_r j_r}$ are disjoint if $i_s \neq j_s$ and $j_s \neq i_t$ for $s \neq t$. Disjoint rotations are “nonconflicting” and therefore commute; it matters neither mathematically nor numerically in which order the rotations are applied. (Disjoint

rotations can therefore be applied in parallel, though that is not our interest here.) Our approach is to take a given sequence of rotations and reorder them into groups of disjoint rotations. The reordered algorithm is numerically equivalent to the original one, but allows a simpler error analysis.

As an example of a rotation sequence already ordered into disjoint groups, consider the following sequence and ordering illustrated for a 6×5 matrix:

$$\begin{bmatrix} \times & \times & \times & \times & \times \\ 1 & \times & \times & \times & \times \\ 2 & 3 & \times & \times & \times \\ 3 & 4 & 5 & \times & \times \\ 4 & 5 & 6 & 7 & \times \\ 5 & 6 & 7 & 8 & 9 \end{bmatrix}.$$

Here, an integer k in position (i, j) denotes that the (i, j) element is eliminated on the k th step by a rotation in the (j, i) plane, and all rotations on the k th step are disjoint. For an $m \times n$ matrix with $m > n$ there are $r = m + n - 2$ stages, and the Givens QR factorization can be written as $W_r W_{r-1} \dots W_1 A = R$, where each W_i is a product of at most n disjoint rotations. It is easy to see that an analogous grouping into disjoint rotations can be done for the scheme illustrated at the start of this section.

Lemma 18.8. *Consider the sequence of transformations*

$$A_{k+1} = W_k A_k, \quad k = 1:r,$$

where $A_1 = A \in \mathbb{R}^{m \times n}$ and each W_k is a product of disjoint Givens rotations. Assume that the individual Givens rotations are performed using computed sine and cosine values related to the exact values defining the W_k by (18.15). Then the computed matrix $\hat{A}_{r+1}$ satisfies

$$\hat{A}_{r+1} = Q^T(A + \Delta A),$$

where $Q^T = W_r W_{r-1} \dots W_1$ and ΔA satisfies the normwise and componentwise bounds

$$\begin{aligned} \|\Delta A\|_F &\leq \gamma_{cr} \|A\|_F, \\ |\Delta A| &\leq m \gamma_{cr} G |A|, \quad \|G\|_F = 1. \end{aligned}$$

(In fact, we can take $G = m^{-1} e e^T$, where $e = [1, 1, \dots, 1]^T$.) In the special case $n = 1$, so that $A = a$, we have $\hat{a}^{(r+1)} = (Q + \Delta Q)^T a$ with $\|\Delta Q\|_F \leq \gamma_{cr}$.

Proof. The proof is analogous to that of Lemma 18.3, so we offer only a sketch. First, we consider the j th column of A , a_j , which undergoes the

transformations $a_j^{(r+1)} = W_r \dots W_1 a_j$. By Lemma 18.7 and the disjointness of the rotations, we have

$$\hat{a}_j^{(r+1)} = (W_r + \Delta W_r) \dots (W_1 + \Delta W_1) a_j,$$

where each ΔW_k depends on j and satisfies $\|\Delta W_k\|_2 \leq \sqrt{2}\gamma_6$. Using Lemma 3.6 we obtain

$$\begin{aligned} \hat{a}_j^{(r+1)} &= Q^T(a_j + d_j), \\ \|d_j\|_2 &\leq ((1 + \sqrt{2}\gamma_6)^r - 1)\|a_j\|_2 = \gamma_{cr}\|a_j\|_2. \end{aligned} \quad (18.16)$$

Hence

$$\|\Delta A\|_F^2 = \sum_{j=1}^n \|d_j\|_2^2 \leq \gamma_{cr}^2 \|A\|_F^2.$$

The inequalities (18.16) for $j = 1:n$ imply that

$$|\Delta A| \leq \gamma_{cr} e(e^T |A|) =: m\gamma_{cr} G|A|,$$

where $\|G\|_F = 1$. The result for $n = 1$ is proved as in Lemma 18.3. II

We are now suitably equipped to give a result for Givens QR factorization.

Theorem 18.9. *Let $\hat{R} \in \mathbf{R}^{m \times n}$ be the computed upper trapezoidal QR factor of $A \in \mathbf{R}^{m \times n}$ ($m \geq n$) obtained via the Givens QR algorithm, with any standard choice and ordering of rotations. Then there exists an orthogonal $Q \in \mathbf{R}^{m \times m}$ such that*

$$A + \Delta A = Q\hat{R},$$

with $\|\Delta A\|_F \leq \gamma_{c(m+n)}\|A\|_F$ and $|\Delta A| \leq m\gamma_{c(m+n)}G|A|$, $\|G\|_F = 1$. (The matrix Q is a product of Givens rotations, the k th of which corresponds to the exact application of the k th step of the algorithm to $\hat{A}_k$.) $\square$

It is interesting that the error bounds for QR factorization with Givens rotations are a factor n smaller than those for Householder QR factorization. This appears to be an artefact of the analysis, and we are not aware of any difference in accuracy in practice.

18.6. Iterative Refinement

Consider a nonsingular linear system $Ax = b$, where $A \in \mathbf{R}^{n \times n}$. Suppose we solve the system using a QR factorization $A = QR$ computed using Householder or Givens transformations (thus, x is obtained by solving $Rx = Q^T b$).

Theorem 18.5, and its obvious analogue for Givens rotations, show that $\hat{x}$ satisfies

$$\|b - A\hat{x}\| \leq p(n)uG(\|A\|\|\hat{x}\| + \|b\|), \quad (18.17)$$

where $\|G\|_F = 1$ and p is a low-degree polynomial. Hence $\hat{x}$ has a normwise backward error of order $p(n)u$. However, since G is a full matrix, (18.17) suggests that the componentwise relative backward error $\omega_{|A|,|b|}(\hat{x})$ need not be small. In fact, we know of no nontrivial class of matrices for which Householder or Givens QR factorization is guaranteed to yield a small componentwise relative backward error.

Suppose that we carry out a step of fixed precision iterative refinement, to obtain $\hat{y}$. The form of the bound (18.17) enables us to invoke Theorem 11.4. We conclude that the componentwise relative backward error $\omega_{|A|,|b|}(\hat{y})$ after one step of iterative refinement will be small as long as A is not too ill conditioned and $|A||\hat{y}|$ is not too badly scaled. This conclusion is similar to that for Gaussian elimination with partial pivoting (GEPP), except that for GEPP there is the added requirement that the LU factorization not suffer large element growth.

Recall from (18.7) that we also have a backward error result in which only A is perturbed. The analysis of §11.1 is therefore applicable, and analogues of Theorems 11.1 and 11.2 hold in which $\eta = u\kappa_\infty(A)$.

The performance of QR factorization with fixed precision iterative refinement is illustrated in Tables 11.1–11.3 in §11.2. The performance is as predicted by the analysis. Notice that the initial componentwise relative backward error is large in Table 11.2 but that iterative refinement successfully reduces it to the roundoff level (despite $\text{cond}(A^{-1})\sigma(A, x)$ being huge). It is worth stressing that the QR factorization yielded a small normwise relative backward error in each example ($\eta_{A,b}(\hat{x}) < u$, in fact), as we know it must.

18.7. Gram–Schmidt Orthogonalization

The oldest method for computing a QR factorization is the Gram–Schmidt orthogonalization method. It can be derived directly from the equation $A = QR$, where $A, Q \in \mathbf{R}^{m \times n}$ and $R \in \mathbf{R}^{n \times n}$ (Gram–Schmidt does not compute the $m \times m$ matrix Q in the full QR factorization and hence does not provide a basis for the orthogonal complement of $\text{range}(A)$). Denoting by a_j and q_j the j th columns of A and Q , respectively, we have

$$a_j = \sum_{k=1}^j r_{kj} q_k.$$

Premultiplying by q_i^T yields, since Q has orthonormal columns, $q_i^T a_j = r_{ij}$, $i = 1:j - 1$. Further,

$$q_j = q'_j / r_{jj},$$

where

$$q'_j = a_j - \sum_{k=1}^{j-1} r_{kj} q_k, \quad r_{jj} = \|q'_j\|_2.$$

Hence we can compute Q and R a column at a time. To ensure that $r_{jj} > 0$ we require that A has full rank.

Algorithm 18.10 (classical Gram–Schmidt). Given $A \in \mathbb{R}^{m \times n}$ of rank n this algorithm computes the QR factorization $A = QR$, where Q is $m \times n$ and R is $n \times n$, by the Gram–Schmidt method.

```

for  $j = 1:n$ 
  for  $i = 1:j-1$ 
     $r_{ij} = q_i^T a_j$ 
  end
   $q'_j = a_j - \sum_{k=1}^{j-1} r_{kj} q_k$ 
   $r_{jj} = \|q'_j\|_2$ 
   $q_j = q'_j / r_{jj}$ 
end

```

Cost: $2mn^2$ flops ($2n^3/3$ flops more than Householder QR factorization with Q left in factored form).

In the classical Gram–Schmidt method (CGS), a_j appears in the computation only at the j th stage. The method can be rearranged so that as soon as q_i is computed, all the remaining vectors are orthogonalized against q_i . This gives the modified Gram–Schmidt method (MGS).

Algorithm 18.11 (modified Gram–Schmidt). Given $A \in \mathbb{R}^{m \times n}$ of rank n this algorithm computes the QR factorization $A = QR$, where Q is $m \times n$ and R is $n \times n$, by the MGS method.

```

 $a_k^{(1)} = a_k, k = 1:n$ 
for  $k = 1:n$ 
   $r_{kk} = \|a_k^{(k)}\|_2$ 
   $q_k = a_k^{(k)} / r_{kk}$ 
  for  $j = k+1:n$ 
     $r_{kj} = q_k^T a_j^{(k)}$ 
     $a_j^{(k+1)} = a_j^{(k)} - r_{kj} q_k$ 
  end
end

```

Cost: $2mn^2$ flops.

It is worth noting that there are two differences between the CGS and MGS methods. The first is the order in which the calculations are performed: in the modified method each remaining vector is updated once on each step instead of having all its updates done together on one step. This is purely a matter of the order in which the operations are performed. Second, and more crucially in finite precision computation, two different (but mathematically equivalent) formulae for r_{kj} are used: in the classical method, $r_{kj} = q_k^T a_j$, which involves the original vector a_j , whereas in the modified method a_j is replaced in this formula by the partially orthogonalized vector $a_j^{(k)}$. Another way to view the difference between the two Gram-Schmidt methods is via representations of an orthogonal projection; see Problem 18.7.

The MGS procedure can be expressed in matrix terms by defining $A_k = [q_1, \dots, q_{k-1}, a_k^{(k)}, \dots, a_n^{(k)}]$. MGS transforms $A_1 = A$ into $A_{n+1} = Q$ by the sequence of transformations $A_k = A_{k-1} R_k$, where R_k is equal to the identity except in the k th row, where it agrees with the final R . For example, if $n = 4$ and $k = 3$,

$$A_3 = [q_1 \quad q_2 \quad a_3^{(3)} \quad a_4^{(3)}] = [q_1 \quad q_2 \quad q_3 \quad a_4^{(4)}] \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & r_{33} & r_{34} \\ 0 & 0 & 0 & 1 \end{bmatrix} = A_4 R_3.$$

Thus $R = R_n \dots R_1$.

The Gram-Schmidt methods produce Q explicitly, unlike the Householder and Givens methods, which hold Q in factored form. While this is a benefit, in that no extra work is required to form Q , it is also a weakness, because there is nothing in the methods to force the computed $\hat{Q}$ to be orthonormal in the face of roundoff. Orthonormality of Q is a consequence of orthogonality relations that are implicit in the methods, and these relations may be vitiated by rounding errors.

Some insight is provided by the case $n = 2$, for which the CGS and MGS methods are identical. Given $a_1, a_2 \in \mathbb{R}^m$ we compute $q_1 = a_1 / \|a_1\|_2$, which we will suppose is done exactly, and then we form the unnormalized vector $q_2 = a_2 - (q_1^T a_2) q_1$. The computed vector satisfies

$$\hat{q}_2 = a_2 - q_1^T (a_2 + \Delta a_2) + \Delta \tilde{q}_2,$$

where

$$|\Delta a_2| \leq \gamma_m |a_2|, \quad |\Delta \tilde{q}_2| \leq u |a_2 - q_1^T (a_2 + \Delta a_2)|.$$

Hence

$$\hat{q}_2 = q_2 + \Delta q_2, \quad |\Delta q_2| \leq \gamma_m |q_1^T| |a_2| + u(|a_2| + (1 + \gamma_m) |q_1^T| |a_2|),$$

and so the normalized inner product satisfies

$$\left| q_1^T \frac{\widehat{q}_2}{\|\widehat{q}_2\|_2} \right| \lesssim (m+2)u \frac{\|a_2\|_2}{\|q_2\|_2} = \frac{(m+2)u}{\sin \angle(a_1, a_2)}, \quad (18.18)$$

where $\angle(a_1, a_2)$ is the angle between a_1 and a_2 . But $\kappa_2(A) \geq \cot \angle(a_1, a_2)$, where $A = [a_1, a_2]$ (Problem 18.8). Hence, for $n = 2$, the loss of orthogonality can be bounded in terms of $\kappa_2(A)$. The same is true in general for the MGS method, as proved by Björck [107, 1967]. A direct proof is quite long and complicated, but a recent approach of Björck and Paige [119, 1992] enables a much shorter derivation; we take this approach here.

The observation that simplifies the error analysis of the MGS method is that the method is equivalent, both mathematically *and numerically*, to Householder QR factorization of the padded matrix $\begin{bmatrix} 0_n \\ A \end{bmatrix} \in \mathbb{R}^{(m+n) \times n}$. To understand this equivalence, consider the Householder QR factorization

$$P^T \begin{bmatrix} 0_n \\ A \end{bmatrix} = \begin{bmatrix} R \\ 0 \end{bmatrix}, \quad P^T = P_n \dots P_2 P_1. \quad (18.19)$$

Let $q_1, \dots, q_n \in \mathbb{R}^m$ be the vectors obtained by applying the MGS method to A . Then it is easy to see that

$$P_1 = I - v_1 v_1^T, \quad v_1 = \begin{bmatrix} -e_1 \\ q_1 \end{bmatrix}, \quad v_1^T v_1 = 2$$

and that the multiplication $A_2 = P_1 \begin{bmatrix} 0_n \\ A \end{bmatrix}$ carries out the first step of the MGS method on A , producing the first row of R and $a_2^{(2)}, \dots, a_n^{(2)}$:

$$A_2 = \begin{matrix} 1 \\ n-1 \\ m \end{matrix} \begin{bmatrix} r_{11} & r_{12} & \dots & r_{1n} \\ 0 & 0 & \dots & 0 \\ 0 & a_2^{(2)} & \dots & a_n^{(2)} \end{bmatrix}.$$

The argument continues in the same way, and we find that

$$P_k = I - v_k v_k^T, \quad v_k = \begin{bmatrix} -e_k \\ q_k \end{bmatrix}, \quad v_k^T v_k = 2, \quad k = 2:n. \quad (18.20)$$

With the Householder–MGS connection established, we are ready to derive error bounds for the MGS method by making use of our existing error analysis for the Householder method.

Theorem 18.12. *Suppose the MGS method is applied to $A \in \mathbb{R}^{m \times n}$ of rank n , yielding computed matrices $\widehat{Q} \in \mathbb{R}^{m \times n}$ and $\widehat{R} \in \mathbb{R}^{n \times n}$. Then there are constants $c_i = c_i(m, n)$ such that*

$$A + \Delta A_1 = \widehat{Q} \widehat{R}, \quad \|\Delta A_1\|_2 \leq c_1 u \|A\|_2, \quad (18.21)$$

$$\|\widehat{Q}^T \widehat{Q} - I\|_2 \leq c_2 u \kappa_2(A) + O((u \kappa_2(A))^2), \quad (18.22)$$

and there exists an orthonormal matrix Q such that

$$A + \Delta A_2 = Q\widehat{R}, \quad |\Delta A_2| \leq c_3 u G|A|, \quad \|G\|_F = 1. \quad (18.23)$$

Proof. To prove (18.21) we use the matrix form of the MGS method. For the computed matrices we have

$$\widehat{A}_k = \widehat{A}_{k+1}\widehat{R}_k + \Delta_k, \quad |\Delta_k| \leq u|\widehat{A}_{k+1}||\widehat{R}_k|.$$

Expanding this recurrence, we obtain

$$A = \widehat{Q}\widehat{R} + \Delta_n\widehat{R}_{n-1}\dots\widehat{R}_1 + \Delta_{n-1}\widehat{R}_{n-2}\dots\widehat{R}_1 + \dots + \Delta_2\widehat{R}_1 + \Delta_1.$$

Hence

$$|A - \widehat{Q}\widehat{R}| \leq u(|\widehat{A}_{n+1}||\widehat{R}_n| \dots |\widehat{R}_1| + \dots + |\widehat{A}_3||\widehat{R}_2||\widehat{R}_1| + |\widehat{A}_2||\widehat{R}_1|), \quad (18.24)$$

and a typical term has the form

$$|\widehat{A}_k||\widehat{R}_{k-1}| \dots |\widehat{R}_1| = |\widehat{q}_1 \dots \widehat{q}_{k-1} \widehat{a}_k^{(k)} \dots \widehat{a}_n^{(k)}| |S_{k-1}|, \quad (18.25)$$

where S_{k-1} agrees with $|\widehat{R}|$ in its first $k-1$ rows and the identity in its last $n-k+1$ rows. Assume for simplicity that $\|\widehat{q}_i\|_2 \equiv 1$ (this does not affect the final result). We have $\widehat{a}_j^{(k+1)} = (I - q_k q_k^T) \widehat{a}_j^{(k)}$, and Lemma 3.8 shows that the computed vector satisfies

$$\|\widehat{a}_j^{(k+1)}\|_2 \leq (1 + 2\gamma_{m+3}) \|\widehat{a}_j^{(k)}\|_2,$$

which implies $\|\widehat{A}_{k+1}\|_F \leq (1 + 2\gamma_{m+3})^k \|A\|_F$ and, from $\widehat{r}_{kj} = fl(\widehat{q}_k^T \widehat{a}_j^{(k)})$, we have $\|\widehat{R}\|_F \leq \sqrt{n}(1 + \gamma_m)(1 + 2\gamma_{m+3})^{n-1} \|A\|_F$. Using (18.24) and exploiting the form of (18.25) we find, after a little working, that

$$\|A - \widehat{Q}\widehat{R}\|_F \leq 4n^2 u \|A\|_F,$$

provided that $(1 + \gamma_m)(1 + 2\gamma_{m+3})^{n-1} < 2$.

To prove the last two parts of the theorem we exploit the Householder–MGS connection. By applying Theorem 18.4 to (18.19) we find that there is an orthogonal $\widetilde{P}$ such that

$$\begin{bmatrix} \Delta A_3 \\ A + \Delta A_4 \end{bmatrix} = \widetilde{P} \begin{bmatrix} \widehat{R} \\ 0 \end{bmatrix} = \begin{bmatrix} \widetilde{P}_{11} \\ \widetilde{P}_{21} \end{bmatrix} \widehat{R}, \quad (18.26)$$

with

$$|\Delta A_i| \leq c_4 u G_i |A|, \quad \|G_i\|_F = 1, \quad i = 3:4.$$

This does not directly yield (18.23), since $\tilde{P}_{21}$ is not orthonormal. However, it can be shown that if we define Q to be the nearest orthonormal matrix to $\tilde{P}_{21}$ in the Frobenius norm, then (18.23) holds with $c_3 = \sqrt{m+1}c_4$ (see Problem 18.11).

Now (18.21) and (18.23) yield $\hat{Q} - Q = (\Delta A_1 - \Delta A_2)\hat{R}^{-1}$, so

$$\|\hat{Q} - Q\|_2 \leq (c_1 + \sqrt{n}c_3)u\|A\|_2\|\hat{R}^{-1}\|_2 \leq \frac{c_5 u \kappa_2(A)}{1 - \sqrt{n}c_3 u \kappa_2(A)},$$

where $c_5 = c_1 + \sqrt{n}c_3$ and we have used (18.23) to bound $\|\hat{R}^{-1}\|_2$. This bound implies (18.22) with $c_2 = 2c_5$ (use the first inequality in Problem 18.13). $\square$

We note that (18.22) can be strengthened by replacing $\kappa_2(A)$ in the bound by the minimum over positive diagonal matrices D of $\kappa_2(AD)$. This follows from the observation that in the MGS method the computed Q is invariant under scalings $A \leftarrow AD$, at least if D comprises powers of the machine base. As a check, note that the bound in (18.18) for the case $n = 2$ is independent of the column scaling, since $\sin \angle(a_1, a_2)$ is.

Theorem 18.12 tells us three things. First, the computed QR factors from the MGS method have a small residual. Second, the departure from orthonormality of Q is bounded by a multiple of $\kappa_2(A)u$, so that Q is guaranteed to be nearly orthonormal if A is well conditioned. Finally, R is the exact triangular QR factor of a matrix near to A in a componentwise sense, so it is as good an R -factor as that produced by Householder QR factorization applied to A . In terms of the error analysis, the MGS method is weaker than Householder QR factorization only in that Q is not guaranteed to be nearly orthonormal.

For the CGS method the residual bound (18.21) still holds, but no bound of the form (18.22) holds for $n > 2$ (see Problem 18.9).

Here is a numerical example to illustrate the behaviour of the Gram-Schmidt methods. We take the 25×15 Vandermonde matrix $A = (p_i^{j-1})$, where the p_i are equally spaced on $[0, 1]$. The condition number $\kappa_2(A) = 1.47 \times 10^9$. We have

$$\text{CGS: } \|A - \hat{Q}\hat{R}\|_2 = 5.0 \times 10^{-16}, \quad \|\hat{Q}^T \hat{Q} - I\|_2 = 3.9,$$

$$\text{MGS: } \|A - \hat{Q}\hat{R}\|_2 = 3.0 \times 10^{-16}, \quad \|\hat{Q}^T \hat{Q} - I\|_2 = 6.2 \times 10^{-9}.$$

Both methods produce a small residual for the QR factorization. While CGS produces a Q showing no semblance of orthogonality, for MGS we have $\|\hat{Q}^T \hat{Q} - I\|_2 \approx \kappa_2(A)u/26$.

18.8. Sensitivity of the QR Factorization

How do the QR factors of a matrix behave under small perturbations of the matrix? This question was first considered by Stewart [944, 1977]. He showed

that if $A \in \mathbf{R}^{m \times n}$ has rank n and

$$A = QR \quad \text{and} \quad A + \Delta A = (Q + \Delta Q)(R + \Delta R)$$

are QR factorization, then, for sufficiently small ΔA ,

$$\frac{\|\Delta R\|_F}{\|R\|_F} \leq c_n \kappa_F(A) \frac{\|\Delta A\|_F}{\|A\|_F}, \quad (18.27)$$

$$\|\Delta Q\|_F \leq c_n \kappa_F(A) \frac{\|\Delta A\|_F}{\|A\|_F}, \quad (18.28)$$

where c_n is a constant. Here, and throughout this section, we use the “economy size” QR factorization with R a square matrix normalized to have nonnegative diagonal elements. Similar normwise bounds are given by Stewart [1951, 1993] and Sun [1971, 1991], and, for AQ only, by Bhatia and Mukherjea [195, 1994] and Sun [1975, 1995].

Componentwise sensitivity analyses have been given by Zha [1126, 1993] and Sun [1972, 1992], [1973, 1992]. Zha’s bounds can be summarized as follows, with the same assumptions and notation as for Stewart’s result above. Let $|\Delta A| \leq \epsilon G|A|$, where G is nonnegative with $\|G\|_\infty = 1$. Then, for sufficiently small ϵ ,

$$\begin{aligned} \frac{\|\Delta R\|_\infty}{\|R\|_\infty} &\leq c_{m,n} \epsilon \operatorname{cond}(R^{-1}) + O(\epsilon^2), \\ \|\Delta Q\|_\infty &\leq c_{m,n} \epsilon \operatorname{cond}(R^{-1}) + O(\epsilon^2), \end{aligned} \quad (18.29)$$

where $c_{m,n}$ is a constant depending on m and n . The quantity $\phi(A) = \operatorname{cond}(R^{-1})$ can therefore be thought of as a condition number for the QR factorization under the columnwise class of perturbations considered. Note that ϕ is independent of the column scaling of A .

As application of these bounds, consider a computed QR factorization $A \approx QR$ obtained via the Householder algorithm, where Q is the computed product of the computed Householder matrices. Theorem 18.4 shows that $A + \Delta A = \tilde{Q}\tilde{R}$, where $\tilde{Q}$ is orthogonal and $|\Delta A| \leq mn\gamma_{cm}G_1|A|$, with $\|G_1\|_F = 1$. Applying Lemma 18.3 to the computation of $\hat{Q}$ we have that $\hat{Q} = \tilde{Q} + \Delta$, where $|\Delta| \leq mn\gamma_{cm}G_2^T|\tilde{Q}|$, with $\|G_2\|_F = 1$. From the expression $Q - \hat{Q} = (Q - \tilde{Q}) + (\tilde{Q} - \hat{Q}) = (Q - \tilde{Q}) - \Delta$ we have, on applying (18.29) to the first term,

$$\|Q - \hat{Q}\|_\infty \leq c_{m,n} u \phi(A) + O(u^2). \quad (18.30)$$

To illustrate this analysis we describe an example of Zha. Let

$$A = \begin{bmatrix} 1 & 1 \\ 0 & \epsilon \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 1/\epsilon \\ 0 & 1/\epsilon \\ 0 & 1/\epsilon \end{bmatrix},$$

where $\epsilon > 0$ is a parameter. It is easy to verify that $A = QR_A$ and $B = QR_B$ are QR factorization (the same Q in each case) where

$$Q = - \begin{bmatrix} 1/\sqrt{2} & 0 \\ 0 & 1 \\ 1/\sqrt{2} & 0 \end{bmatrix}.$$

With $\epsilon = 10^{-8}$, the computed Q factors from MATLAB are

$Q_A =$ $\begin{bmatrix} -7.0711\text{e-}001 & 1.2539\text{e-}008 \\ 0 & -1.0000\text{e+}000 \\ -7.0711\text{e-}001 & -1.2539\text{e-}008 \end{bmatrix}$	$Q_B =$ $\begin{bmatrix} -7.0711\text{e-}001 & 1.5872\text{e-}016 \\ 0 & -1.0000\text{e+}000 \\ -7.0711\text{e-}001 & -1.5872\text{e-}016 \end{bmatrix}$
--	--

and

$$\|Q - Q_A\|_2 \approx 1.8 \times 10^{-8}, \quad \|Q - Q_B\|_2 \approx 2.2 \times 10^{-16}.$$

Since $\phi(A) \approx 2.8 \times 10^8$ and $\phi(B) \approx 3.8$, the actual errors match the bound (18.30) very well. Note that $\kappa_2(A) \approx 2.8 \times 10^8$ and $\kappa_2(B) \approx 2.1 \times 10^8$, so the normwise perturbation bound (18.28) is not strong enough to predict the difference in accuracy of the computed Q factors, unless we scale the factor $1/\epsilon$ out of the last column of B to leave a well-conditioned matrix. The virtue of the componentwise analysis is that it does not require a judicious scaling in order to yield useful results.

18.9. Notes and References

The earliest appearance of Householder matrices is in the book by Turnbull and Aitken [1029, 1932, pp. 102–105]. These authors show that if $\|x\|_2 = \|y\|_2$ ($x \neq -y$) then a unitary matrix of the form $R = azz^* - I$ (in their notation) can be constructed so that $Rx = y$. They use this result to prove the existence of the Schur decomposition. The first systematic use of Householder matrices for computational purposes was by Householder [586, 1958], who used them to construct the QR factorization. Householder's motivation was to compute the QR factorization with less arithmetic operations (in particular, less square roots) than are required by the use of Givens rotations.

In the construction of §18.1 for a Householder matrix P such that $Px = \sigma e_1$, the other choice of sign, $\text{sign}(\sigma) = \text{sign}(x_1)$, can be used, provided that v_1 is formed in a numerically stable way. The appropriate formula is derived as follows [264, 1976], [819, 1971], [820, 1980, p. 91]:

$$v_1 = x_1 - \text{sign}(x_1)\|x\|_2 = \frac{x_1^2 - \|x\|_2^2}{x_1 + \text{sign}(x_1)\|x\|_2} = \frac{-(x_2^2 + \cdots + x_n^2)}{x_1 + \text{sign}(x_1)\|x\|_2}.$$

A detailed analysis of different algorithms for constructing P such that $Px = \sigma e_1$ is given by Parlett [819, 1971].